7. Bölüm
Fonksiyonlar

7.1 Fonksiyon Nedir?

Yapısal Programlama mantığında çözülecek problem genel olarak fonksiyonların birbirini çağırmasıyla yapıldığı 2.7 başlığında anlatılmıştı. Yani kodlaması yapılacak işi birbirini çağıran fonksiyonlar yazarak ana fonksiyondan bu fonksiyonları çağırarak yaparız. Ayrıca yazılacak programda birbiriyle ilgili fonksiyonlar bir araya toplanarak ayrı bir dosyada tutulur. Bu durum 5.1 başlığı altında anlatılmıştı.

Bu bölüme kadar gördüğümüz kodlarda çözümü oluşturan tüm tasarım parçaları **ana fonksiyon** (main function) içerisine çözülmüştür. Bir bilgisayar programı, genel olarak, tek başına tüm görevleri yerine getiren tek bir main fonksiyonundan oluşmaz. Bu durumda ana problemin büyüklüğüne göre ana fonksiyon bloğumuz uzar, okunabilirliği azalır, müdahale etmek zorlaşır. Bu tip büyük programları kendi içerisinde, her biri özel bir işi çözmek için tasarlanmış parçacıklarından oluşturmak daha mantıklı olacaktır. Yani **böl ve yönet** (divide and conquer) tekniği uygulanır. Çünkü büyük bir problemin toptan çözülmesi, problemin parçalar halinde çözülmesinden her zaman daha yorucu ve zordur. Yazılım geliştirmede problemi büyük ana parçalara ayırırız. Daha sonra bu büyük parçaları daha küçük parçalara ayırarak çözeriz. Buna **yukarıdan aşağıya** (top down) yaklaşım adı verilir.

C dilindeki gibi C++ dilinde de fonksiyonlar;

- Fonksiyonlar, belirli bir görevi gerçekleştiren bir kod bloğudur.
- Parametreli alabilir, girdilerle ya da parametresiz bir şeyler yapar ve ardından cevabı verebilir. Bilgiyi fonksiyona argümanlar ile aktarırız.
- Her fonksiyonun bir kimliği vardır.
- Bir fonksiyon program içerisinde istenildiği kadar farklı yerlerden ulaşılarak çalıştırılabilir ya da çağrılabilir. Bir başka deyişle tekrar kullanılabilir.
- Bir fonksiyon, çağıran koda bir değer döndürebilir.
- Bir fonksiyon, bir başka fonksiyondan çağrılabilir.

C++ dilinde iki tip fonksiyon bulunmaktadır; başlık dosyalarında bize sunulan **hazır fonksiyonlar** ve programcının tanımlayacağı **kullanıcı tanımlı fonksiyonlar** (user defined function). Hazır fonksiyonlar, programcı için hazırlanmış ve her işletim sistemi için derlenmiş ve test edilmiş modüllerde (başlık dosyalarında) yer alır. Yani bir modül içindeki öğeler tek bir amaç etrafında birlikte çalışacak şekilde belirlenir. Bu belirleme işleminde **yüksek uyum** (high cohesion) ya da tutarlılık aranır. Yazılım modülleri arasındaki karşılıklı **düşük bağlaşımın** (low coupling) olması istenir. Yani modüller birbirine en az bağımlılık ile hazırlanmalıdır.

7.2 Hazır Fonksiyonlar

C++ geliştiricileri, programcıların kullanmaları için, önceden yazılmış olan hazır fonksiyonlar hazırlamışlardır. Hazır fonksiyonlar teknik olarak C++ dilinin parçası değillerdir. Yalnızca standart hale getirilmişlerdir. Programcı, kullanmak istediği hazır fonksiyon hangi modülde ise o modüle ilişkin **başlık** (header) dosyasını `#include` **ön işlemci yönergesi** (preprocessor directive) ile kendi projesine dahil eder. Aslında bu başlık

Kütüphane	Örnek Fonksiyon	Açıklama
`<cmath>`	`sqrt(x)`	x sayısının karekökünü hesaplar.
`<cstdlib>`	`rand(x)`	0 ile x arasında rastgele bir sayı üretir.
`<algorithm>`	`sort(b, e)`	Belirtilen aralığı sıralar.
`<string>`	`getline()`	Boşluk içeren metin satırını akıştan okur.
`<cctype>`	`isdigit(c)`	Karakterin rakam olup olmadığını denetler.

Tablo 7.1: Sık Kullanılan Standart C++ Fonksiyonlarından Bazıları

dosyalarında;

- Hazır fonksiyonların sadece **prototipleri** (prototype), başlık (header) dosyalarında bulunur. Prototip derleyiciye, parametreleri ve geri dönüş değeri şu olan bir fonksiyon var demenin yoludur.
- Fonksiyonların kendileri yani derlenmiş halleri her işletim sistemi için ayrı derlenmiş olan kütüphane (library) dosyaları içerisinde bulunur.
- Derleme işlemi sırasında **bağlayıcı** (linker) program tarafından bu fonksiyonlar icra edilebilir dosyaya

(executable file) dahil edilir.

§7.2.1 cmath Başlık Dosyası

Matematik işlemleri gerçekleştirmek için standart hale getirilmiş bir başlık dosyasıdır. Tablo 7.2'de fonksiyon listesi verilmiştir.

Kategori	Fonksiyon	Açıklama
Üstel ve Log.	`pow(x, y)`	x^y değerini hesaplar.
	`exp(x)`	e^x değerini hesaplar.
	`sqrt(x)`	x sayısının karekökünü hesaplar.
	`log10(x)`	x sayısının 10 tabanında logaritmasını ($\log x$) hesaplar.
	`log(x)`	x sayısının doğal logaritmasını ($\ln x$) hesaplar.
Yuvarlama	`ceil(x)`	x'i kendisinden büyük veya eşit en küçük tam sayıya yuvarlar.
	`floor(x)`	x'i kendisinden küçük veya eşit en büyük tam sayıya yuvarlar.
	`round(x)`	x'i en yakın tam sayıya yuvarlar.
Trigonometrik	`sin(x)`	Radyan cinsinden verilen açının sinüsünü hesaplar.
	`cos(x)`	Radyan cinsinden verilen açının kosinüsünü hesaplar.
	`tan(x)`	Radyan cinsinden verilen açının tanjantını hesaplar.
Mutlak Değer	`abs(x)`	Tam sayıların mutlak değerini döndürür.
	`fabs(x)`	Ondalıklı sayıların mutlak değerini döndürür.
Diğer	`fmod(x, y)`	x/y işleminin kalanını (ondalıklı) döndürür.
	`hypot(x, y)`	$\sqrt{x^2+y^2}$ (hipotenüs) değerini hesaplar.

Tablo 7.2: <cmath> Kütüphanesi Temel Fonksiyonları

Örnek olarak kenarları verilen bir üçgenin alanını hesaplayan bir program aşağıda verilmiştir.

```
#include <iostream>
#include <cmath>
int main() {
    int kenar1,kenar2,kenar3;
    double alan,kenarlarToplamininYarisi;
    std::cout << "Üçgenin Kenarlarını Giriniz:";
    std::cin >> kenar1 >> kenar2 >> kenar3;
    if ((kenar1+kenar2 <= kenar3) ||
        (kenar2+kenar3 <= kenar1) ||
        (kenar1+kenar3 <= kenar2) ) {
            std::cout << "Böyle bir üçgen olamaz." << endl;
            return 1;
    }
    kenarlarToplamininYarisi=(kenar1+kenar2+kenar3)/2.0;
    alan=sqrt( kenarlarToplamininYarisi*
        (kenarlarToplamininYarisi-kenar1)*
        (kenarlarToplamininYarisi-kenar2)*
        (kenarlarToplamininYarisi-kenar3) );
    std::cout << "Ücgenin alanı: " << alan ;
}
```

§7.2.2 cstdlib Başlık Dosyası

Bu başlıkta;

- Alfa sayısal metinleri ilkel veri tiplerine veya bunun tersini yapan tür dönüşüm fonksiyonları (`atoi()`, `itoa()`, `atof()`, `strtod()`, ...),
- **Çalışma anında** (run-time) bellek tahsisi ve tahsisli belleğin serbest bırakılmasını sağlayan hafıza yerleşim fonksiyonları (`malloc()`, `free()` ...)
- Rastgele sayı üretme fonksiyonları (`rand()`, `srand()`) bulunur.

Şimdilik rastgele sayı üretme üzerinde durulacaktır. Aşağıda bir zar için rastgele sayı üreten bir program verilmiştir.

```
#include <iostream>
#include <cstdlib>
int main() {
    int rastgeleZar;
    rastgeleZar=1 + rand() %6; /* std::rand() fonksiyonunun üreteceği sayıyı kalan işleci ile 6
     ↪ sayısına     böleriz ki 0 ile 5 arasında bir sayı elde edelim. Buna da 1 eklersek zarın
     ↪ rakamları ortaya çıkar. */
    std::cout << "Atılan Zar:" << rastgeleZar;
}
```

Fonksiyon	Açıklama
`int rand();`	`0` ile `RAND_MAX` yani `32767` arasında rastgele bir sayı üretir.
`int srand(unsigned int);`	`rand()` fonksiyonu belli bir başlangıç değerinden itibaren bir dizi matematiksel işlem sonucu rastgele bir değer üretir. `srand()` fonksiyonu, `rand()` fonksiyonuna başlangıç değerini farklı vermek için kullanılır.

Tablo 7.3: Stdlib.h Başlık Dosyasının Rastgele Sayı Fonksiyonları

Fakat programın her çalışmasında `rand()` aynı başlangıç değerini kullandığından **aynı rastgele değerleri üretir**. Bu nedenle aynı zar rakamı gelmiş olur. Bunu değiştirmek için `srand()` fonksiyonu kullanılır.

```
#include <iostream>
#include <cstdlib>
int main() {
    unsigned randBaslangicDegeri;
    std::cout << "std::rand() için başlangıç değeri olabilecek bir sayı giriniz:";
    std::cin >> randBaslangicDegeri;
    srand(randBaslangicDegeri); /* std::rand() fonksiyonunun üreteceği sayı her zaman aynı
     ↪ başlangıç değeriyle başladığından her program çalıştığında üterilecek ilk rastgele sayı
     ↪ ve sonrasındaki sayılar hep olur. Bu hesaplamanın bir başka başlangıçla başlaması
     ↪ üterilecek ilk rastgele sayı ve sonraski üretilecekleri değiştirir. Bunu değiştirmenin
     ↪ yolu std:srand() fonksiyonunu kullanmaktır. */
    int rastgeleZar;
    rastgeleZar=1 + rand() %6;
    std::cout << "Atılan Zar:" << rastgeleZar;
}
```

Rastgele sayı her seferinde kullanıcıdan alınan sayıya göre hesaplandığından her seferinde farklı bir zar rakamı üretilmiş olacaktır. **Ancak başlangıç değerinin her seferinde kullanıcıdan alınması bir başka problemdir**. Bunun üstesinden gelmek için bilgisayarın saatini sayı olarak veren `<ctime>` başlık dosyasındaki `time()` fonksiyonu `nullptr` parametresiyle kullanılabilir.

```
#include <iostream>
#include <cstdlib>
#include <ctime>
int main() {
    unsigned randBaslangicDegeri=time(nullptr);
    /* std:srand() fonksiyonu ile std::rand() fonksiyonu ilk değer verilir. Verilecek ilk değer
     ↪ std::time() fonksiyonuna nullptr verilerek sistem zamanından gelen değer
     ↪ kullanılır.Böylece her çalıştırmada farklı rastgele sayı üretilmesi sağlanacaktır. */
    srand(randBaslangicDegeri); // srand(time(nullptr)); olarak da yazılabilir.
    int rastgeleZar;
    rastgeleZar=1 + rand() %6;
    std::cout << "Atılan Zar:" << rastgeleZar;
}
```

7.3 Kullanıcı Tanımlı Fonksiyonlar

Programcılar kendi fonksiyonlarını oluşturarak kodlarını daha kullanışlı hale getirirler. Bu tür fonksiyonlara **kullanıcı tanımlı fonksiyonlar** (user defined function) denilir.

Şimdiye kadar yazdığımız kodlarda ana fonksiyonumuz olan `main()` hep `sqrt()`, `rand()` gibi fonksiyonları çağırmıştır. Bir fonksiyonu çalıştıran koda, **çağıran program** adı verilir. Hazır fonksiyonların işlendiği 7.2 başlığındaki örneklerde `main()` ana fonksiyonumuz hep çağıran programdır.

Kullanıcı tanımlı fonksiyonu oluşturmak ve kullanmak için bu dört unsuru bilmemiz gerekir;

1. **Fonksiyon Tanımı** (function definition): Bir fonksiyonun girdileri, yaptığı işlem ve döndüreceği değeri içerecek şekilde **başlık ve gövdesinin kodlanmasından oluşur**. Fonksiyon tanımı, fonksiyonu derleme yapılabilmesi için eksiksiz bir şekilde tamamlar. **Başlık**, fonksiyonun döndüreceği veri tipi, fonksiyon kimliği ve aç kapa parantez içindeki parametre listesinden oluşur. **Gövde** ise süslü parantezler ve içindeki fonksiyonun taptığı işi içeren kodlardır.
2. **Fonksiyon Bildirimi** (function declaration): Derleyiciye, programın ilerleyen kısımlarında (veya belki başka bir dosyada) verilen kimlikte ve verilen parametrelere sahip bir fonksiyonun var olduğunu söyler. Derleyici bu sayede, fonksiyonun gövdesini görmese bile onun nasıl çağrılacağını, dönüş tipi ve argümanlarını bilir. Bildirimin amacı, derleyiciye "Böyle bir fonksiyon var " bilgisini vermektir. Bildirim yapılıp, tanımı yapılmayan bir bir fonksiyona sahip program derlenmeye çalışıldığında, bağlama zamanında (link-time) hata alırız.
3. **Fonksiyon Prototipi** (function prototype): Aslında fonksiyon bildirimi ile aynı şeydir. Ancak prototip terimi, genellikle fonksiyonun kodun en tepesinde, `main()` fonksiyonundan önce yapılan bildirimini ifade etmek için kullanılır.
4. **Fonksiyon Çağırma**(function call/invoke function): Çağıran program içinde, bildirimi daha önce yapılmış bir fonksiyon, kimliğini ve ardından parantez içindeki argüman listesini yazarak çağırabiliriz.

§7.3.1 Fonksiyon Tanımı Nasıl Yapılır?

Fonksiyon tanımı (function definition), fonksiyonun işleyip **döndüreceği veri tipini**, **kimliği**, işleyeceği bağımsız değişkenleri temsil eden **parametre listesini**, içerecek şekilde **başlığının** ve **gövdesinin** kodlanmasından oluşur. Aşağıda bir fonksiyon tanımının sözde kodu verilmiştir;

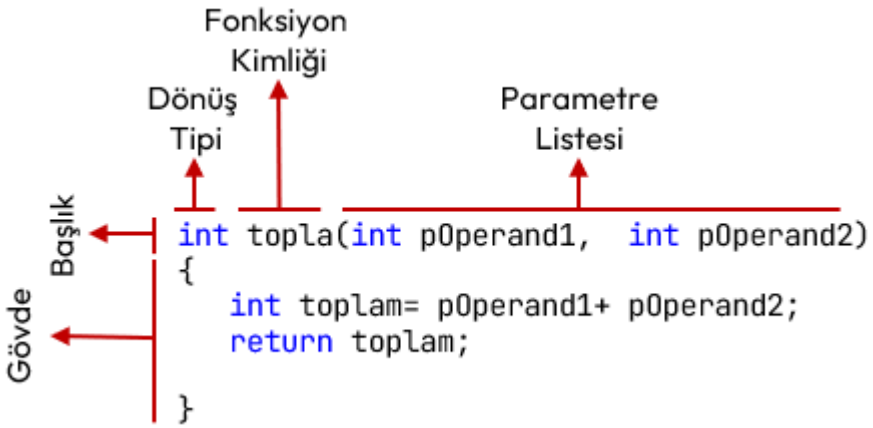
```
geri-dönüş-tipi fonksiyon-kimliği(parametre-listesi) //BAŞLIK
{ //GÖVDE BAŞLANGICI
 //...
 // Algoritma
 //...
 return geri-dönüş-değeri;
 //...
} //GÖVDE BİTİŞİ
```

Sözde kodu şu şekilde açıklayabiliriz;

- **Fonksiyonun kimliği benzersizdir**. Aynı zamanda **fonksiyon adı** (function name) olarak adlandırılır. Fonksiyon kimliklendirmede, 4.3.5 başlığındaki değişken kimliklendirme kuralları geçerlidir.
- Fonksiyon kimliğinden sonra **parametreleri barındıracak açıp kapatılan parantez** olmalıdır. Bu durum değişken tanımından fonksiyon tanımlarını ayırmak içindir.
- Bağımsız değişkenler, parametre olarak adlandırılır;
  - **Parametre listesi**, bir fonksiyonun parametrelerinin veri tipini, sırasını ve sayısını belirtir.
  - Parametreler isteğe bağlıdır, yani bir fonksiyon hiçbir parametre içermeyebilir.
  - **Her bir parametreye kimlik verilmelidir**. Çünkü fonksiyon gövdesinde bu değişkenler işlenir.
  - Parametreler birbirinden **virgül işleci** (comma operator) ile ayrılır.
- Parametreler fonksiyon **gövdesinde** yapılacak işlere temel oluşturur;
  - Gövde, fonksiyonun ne yaptığını tanımlayan ve süslü parantezler arasında yer alan talimatları içerir.
  - Gövdede, **yapısal programlama yapıldığından**, ilk önce olacaksa değişkenler tanımlanmalı daha sonra parametrelerle birlikte bu değişkenleri işleyecek talimatlar yazılmalıdır.
  - Gövdede, fonksiyonda istenilen sonuca ulaşmak için kullanacağı adımlara karar verilir yani **algoritması belirlenir**.
- İşlenen parametrelerden ve gövde içerisinde tanımlanan değişkenlerden bir **geri dönüş değeri** hesaplanabilir. Değer hesaplanmış ise `return geri-dönüş-değeri;` talimatıyla fonksiyonun icrası sonlandırılır. textttreturn Talimatı çoğunlukla fonksiyonundaki son talimat olarak görünür ama fonksiyon gövdesinde herhangi bir yerden geri dönülebilir.
- `return` Talimatıyla geri döndürülen değerin tipi, **fonksiyonun geri dönüş tipidir**.
- Bazı durumlarda fonksiyon, geri değer döndürmeyebilir;
  - Bu durunda fonksiyonun **geri dönüş tipi yoktur** ve bu nedenle geri dönüş tipi`void` olarak belirtilir.
  - Geri dönüş tipi `void` olan fonksiyonun icrası `return;` talimatıyla sonlandırılır.
  - Eğer `return;` talimatı hiç yazılmaz ise süslü parantez kapatılıncaya kadar fonksiyon icra edilir

ve fonksiyondan geri dönülür.

> Bir de fonksiyonun çağrıldığı yerde, **fonksiyonun döndürdüğü değerin tipi ile eşleşen bir değişkene dönüş değeri atanabilir**. Zorunlu değildir.
> Ayrıca C++ dilinde **bir fonksiyon gövdesinde bir başka fonksiyon tanımlanamaz!**



Şekil 7.1: Fonksiyon Tanımı Örneği

Şekil 7.1'de; iki parametresini toplayıp, toplamı geri döndüren bir `topla` fonksiyonunun tanımlaması verilmiştir. Aşağıdaki şekilde açıklayabiliriz;

- Fonksiyonun kimliği `topla` olarak belirlenmiştir;
- `topla` Fonksiyonunun parametre listesinde iki adet `int` veri tipinde parametre (`pOperand1`, `pOperand2`) tanımlanmıştır.
- `topla` Fonksiyonu gövdesinde yapısal programlama yapıldığından ilk önce `toplam` yerel değişkeni tanımlandı. Daha sonra parametrelerin toplamı `toplam` değişkenine atandı.
- Son olarak `return toplam;` talimatı ile hesaplan değer geri döndürülmüştür.

Aşağıda bu `topla` fonksiyonu kullanan bir program örneği verilmiştir;

```
#include <iostream>

/* 1. Fonksiyon Tanımlama (Function Definition): */
int topla(int pOperand1,int pOperand2) //Fonksiyon başlığı
{ //Fonksiyon Gövdesi başlangıcı
    int toplam=pOperand1+ pOperand2;
    return toplam; // doğrudan "return pOperand1+ pOperand2;" da yazılabilirdi.
} //Fonksiyon Gövdesi bitişi: Sonunda noktalı virgül OLMAZ!
/* Bu noktadan sonra "topla" fonksiyonu her yerden çağrılabilir! */

/* 2. ÇAĞRI ORTAMI: main fonksiyonu içinde bu fonksiyon çağrılacak: */
int main () {
    int sonuc;

    /* 3. topla fonksiyonu 1 ve 2 argümanlarıyla çağrılıyor ama geri döndürdüğü değer
     ↪ kullanılmıyor. */
    topla(1,2);
    /* 4. topla fonksiyonu 2 ve 3 argümanlarıyla çağrılıyor. Geri döndürdüğü değer std::cout
     ↪ içine argüman olarak giriyor. */
    std::cout << "1. toplam:" <<  topla(2,3) << std::endl;

    /* 5. topla fonksiyonu 3 ve 4 argümanlarıyla çağrılıyor. Geri döndürdüğü değer sonuc
     ↪ değişkenine atanıyor.*/
    sonuc= topla(3,4);
    std::cout << "2. toplam:" << sonuc << std::endl;

    /* 6. topla fonksiyonu iki kez çağrılıyor. Her iki çağrıdaki geri dönüş değeri gecici bir
     ↪ yerde saklanıp toplamı sonuc değişkenine atanıyor. */
    sonuc= topla(1,2)+topla(3,4);
```

```
    std::cout << "3. toplam:" << sonuc << std::endl;

    /* 6. topla fonksiyonu üç kez çağrılıyor. İlk çağrıdaki pOperand1 parametresine topla(2,3)
     ↪  fonksiyonunun sonucu argüman olarak geçiriliyor. */
    sonuc= topla( topla(1,2) ,3)+topla(4,5);
    std::cout << "4. toplam: " << sonuc << std::endl;
}

/* Program Çıktısı:
1. toplam:5
2. toplam:7
3. toplam:10
4. toplam: 15
*/
```

Örnek program, aslında 2.7 başlığında bahsedilen yapısal programlamanın çerçevesidir. Ana program içinde problemin çözümü, çeşitli fonksiyonların birbirini çağırması ile yapılmıştır. Fonksiyon içinde işlenecek veriye ilişkin değişken tanımlanmış ve bu veriler ile veri yapılarını işleyecek kontrol işlemleri (bizim örneğimizde kontrol işlemi yok) birbirinden ayrılmıştır. **Okunaklılık çok yüksek bir kod elde edilmiştir.**

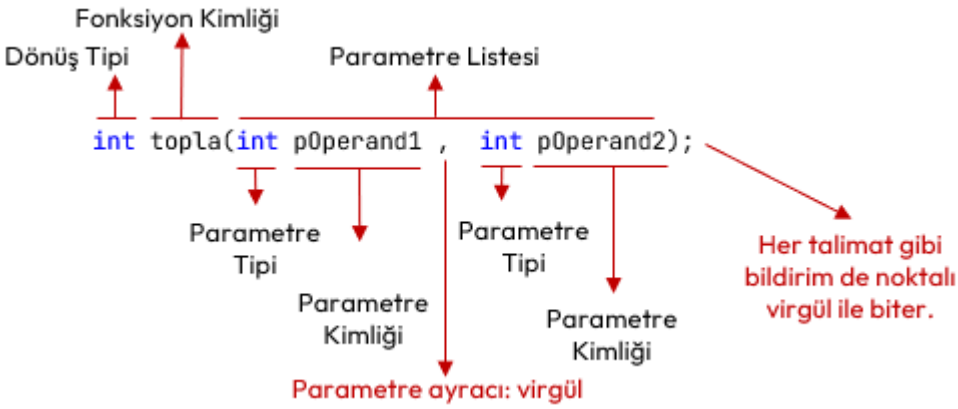
§7.3.2 Fonksiyon Bildirimi Nasıl Yapılır?

Fonksiyon bildirimi (function declaration), derleyiciye, programın ilerleyen kısımlarında (belki başka bir dosyada) belirtilen kimlikte ve verilen parametrelere sahip bir fonksiyonun var olduğunu söyler. Aşağıda bir fonksiyon bildiriminin sözde kodu verilmiştir;

```
geri-dönüş-tipi fonksiyon-kimliği(parametre-listesi);
```

Fonksiyon bildirimi, fonksiyon tanımındaki **başlık kısmı gibidir**;

- Bildirimin, parametre listesinde, **parametre kimlikleri zorunlu değildir.**
- Bildirim, talimatlar gibi **noktalı virgül ile biter**.
- Bildirimi yapılan fonksiyonun, fonksiyon tanımı (function definition) bir yerde mutlaka yapılmalıdır!
- Bildirimi yapılmış fonksiyonun tanımında aynı kimlik kullanılıdır.



Şekil 7.2: Fonksiyon Bildirimi Örneği

Şekil 7.2'de iki adet `int` veritipinde parametresi olan ve `int` veri tipinde geri değer döndüren `topla` kimlikli bir fonksiyon bildirimi örneği verilmiştir. Bildirim örneğini aşağıdaki şekilde açıklayabiliriz;

- Fonksiyonun kimliği `topla` olarak belirlenmiştir;
- `topla` Fonksiyonunun parametre listesinde iki adet `int` veri tipinde parametre (`pOperand1`, `pOperand2`) tanımlanmıştır. Parametre kimlikleri verilmeyebilirdi. Yani `int topla(int,int);` aynı bildirimdir.
- Bildirimde `topla` fonksiyonunun geri dönüş veri tipi, `int` olarak belirlenmiştir.

Fonksiyon tanımının yapıldığı 7.3.1 başlığındaki `topla` örneğimizi önce bildirim yapılarak aşağıdaki şekilde yazılabilir;

```
#include <iostream>

/* 1. Fonksiyon Bildirimi (Function Declaration): */
int topla(int, int);
/* Bu noktadan sonra bu bildirim ile topla fonksiyonu çağrılabilir ancak tanımı sonradan
 ↪  yapılmalıdır! */
```

```
/* 2. ÇAĞRI ORTAMI: main fonksiyonu içinde bu fonksiyon çağrılacak: */
int main () {
    int sonuc;

    /* 3. topla fonksiyonu 1 ve 2 argümanlarıyla çağrılıyor ama geri döndürdüğü değer
     ↪ kullanılmıyor. */
    topla(1,2);
    /* 4. topla fonksiyonu 2 ve 3 argümanlarıyla çağrılıyor. Geri döndürdüğü değer std::cout
     ↪ içine argüman olarak giriyor. */
    std::cout << "1. toplam:" <<  topla(2,3) << std::endl;

    /* 5. topla fonksiyonu 3 ve 4 argümanlarıyla çağrılıyor. Geri döndürdüğü değer sonuc
     ↪ değişkenine atanıyor.*/
    sonuc= topla(3,4);
    std::cout << "2. toplam:" << sonuc << std::endl;

    /* 6. topla fonksiyonu iki kez çağrılıyor. Her iki çağrıdaki geri dönüş değeri gecici bir
     ↪ yerde saklanıp toplamı sonuc değişkenine atanıyor. */
    sonuc= topla(1,2)+topla(3,4);
    std::cout << "3. toplam:" << sonuc << std::endl;

    /* 6. topla fonksiyonu üç kez çağrılıyor. İlk çağrıdaki pOperand1 parametresine topla(2,3)
     ↪ fonksiyonunun sonucu argüman olarak geçiriliyor. */
    sonuc= topla( topla(1,2) ,3)+topla(4,5);
    std::cout << "4. toplam: " << sonuc << std::endl;
}

/* 7. Fonksiyon Tanımlama (Function Definition): */
int topla(int pOperand1,int pOperand2) //BAŞLIK
{ //GÖVDE Blok Başlangıcı
    return pOperand1+ pOperand2;
} //GÖVDE Blok Bitişi
```

Fonksiyon tanımının yapıldığı 7.3.1 başlığındaki gibi **hiç bildirim yapılmadan da fonksiyonlar tanımlanabilir**. Ancak fonksiyon tanımlandığı yerden sonra bildirimi yapılmış sayılıp çağrılabilir!

Aşağıdaki fonksiyon bildirimi içeren bir başka programı inceleyelim. Bu program derleyici tarafından derlenir ancak **bağlama** (link) aşamasında bağlayıcı program (linker) hata verir. Çünkü **fonksiyon tanımlamaları** (function definition) yapılmamıştır!

```
/* 1. Fonksion Bildirimleri (Function Declaration): */
/* "ikiSayiyiTopla" kimlikli bir fonksiyon olduğu ve iki tam sayı parametre aldığı ve ayrıca bir
 ↪ tam sayı geri döndürdüğü derleyiciye bildiriliyor. */
int ikiSayiyiTopla(int,int);

/* "sayiyiKonsolaYaz" kimlikli bir fonksiyon olduğu ve bu fonksiyonun bir tam sayı parametre
 ↪ aldığı ve geri değer döndürmediği derleyiciye bildiriliyor. */
void sayiyiKonsolaYaz(int);

/* "konsoldanTamsayiOku" kimlikli bir fonksiyon olduğu ve bu fonksiyonun bir parametre almadığı
 ↪ ve bir tam sayı geri döndürdüğü derleyiciye bildiriliyor. */
int konsoldanTamsayiOku();

/* 2. ÇAĞRI ORTAMI: main fonksiyonu içinde bu fonksiyonlar çağrılacak: */
int main() {
    int sayi;
    sayi=konsoldanTamsayiOku();
    sayiyiKonsolaYaz(13); //sayiyiKonsolaYaz fonksiyonu 13 argümanı ile çağrıldı
    sayiyiKonsolaYaz(sayi); //sayiyiKonsolaYaz fonksiyonu sayi argümanı ile çağrıldı
    int toplam= ikiSayiyiTopla(sayi,20);  //ikiSayiyiTopla fonksiyonu sayi ve 20 argümanları ile
     ↪ çağrıldı
```

```
    //...
}
```

Program ana fonksiyondan başladığından her programcı ana fonksiyona odaklanır. Bu nedenle **ilk önce fonksiyon bildirimleri yapılıp fonksiyon tanımlarının ana fonksiyon sonrasına bırakılması etik bir davranıştır**.

Aşağıda pozitif sayı girilmesini garanti ettikten sonra sayının 10 sayısına bölünebilirliğini bulan program verilmiştir.

```
#include <iostream>
/* 1. Fonksion Bildirimleri (Function Declaration): */
int pozitifSayiOku(); // parametre almayan bir fonksiyon bildirimi.
int onaBolunurMu(int);
/* 2. ÇAĞRI ORTAMI: main fonksiyonu içinde bu fonksiyonlar çağrılacak: */
int main () {
    int birPozitifSayi=pozitifSayiOku();
    if (onaBolunurMu(birPozitifSayi))
        std::cout << "Girilen Pozitif Sayı 10 ile bölünebilir.";
    else
        std::cout << "Girilen Pozitif Sayı 10 ile tam bölünmez.";
}
/* 3. Fonksiyon Tanımlamaları (Function Definition): */
int pozitifSayiOku() {
    int okunan;
    do {
        std::cout << "Lütfen pozitif sayı giriniz: ";
        std::cin >> okunan;
        if (okunan<=0)
            std::cout << "HATA:Pozitif Sayı GİRMEDİNİZ!" << std::endl;
    } while (okunan <= 0);
    return okunan;
}
int onaBolunurMu(int p){
    return (p%10)==0;
}
```

Bunların dışında hiçbir değer geri döndürmeyen fonksiyonlar da tanımlayabiliriz. Bunlara diğer dillerde **prosedür** (procedure) adı da verilir. Bu durumda değeri olmayan veri tipi olan `void` anahtar kelimesi kullanılır. Bu tür fonksiyonların da geri dönüş talimatı yalnızca `return;` şekilde yazılır.

```
#include <iostream>
/* 1. Fonksion Bildirimleri (Function Declaration): */
void printMenu(); // geri değer döndürmeyen fonksiyon bildirimi.
/* 2. ÇAĞRI ORTAMI: main fonksiyonu içinde bu fonksiyonlar çağrılacak: */
int main() {
    int secim;
    do {
        printMenu();
        std::cin >> secim;
        switch(secim) {
        case 1:
            std::cout << "Verileri Yazdım..." << std::endl;
            break;
        case 2:
            std::cout << "Verileri Okudum..." << std::endl;
            break;
        case 0:
            std::cout << "Programdan Çıkılıyor..." << std::endl;
            break;
        default:
            std::cout << "Tekrar Seçim Yapınız!" << std::endl;
        }
```

```
    } while (secim!=0);
}
/* 3. Fonksiyon Tanımlamaları (Function Definition): */
void printMenu() {
    std::cout << std::endl;
    std::cout << "1-Verileri Yaz." << std::endl;
    std::cout << "2-Verileri Oku." << std::endl;
    std::cout << "0-ÇIKIŞ." << std::endl;
    return;
}
```

Tek Tanım Kuralı:

- Bildirimi yapılan bir fonksiyon, bir başlık (header) dosyasında olabilir ve bu **başlık dosyası birden çok kaynak kodumuza dahil edilebilir**.
- Bildirimi yapılan bir fonksiyon, derleyici tarafından **dahil edileceği kaynak dosyaların tamamında yalnızca bir kez tanımlanabilir (one definition rule)**. Kısaca birden çok dosya için derleme söz konusu olduğunda birden çok bildirim olabilir ama bir adet fonksiyon tanımı olmalıdır.
- İleride **başlık koruması** (header guard) kavramının detayı verilecektir.

7.4 Çağrı Kuralı

Çağrı kuralı (calling convention), bir fonksiyon çağrısıyla karşılaşıldığında argümanların fonksiyona hangi sıraya geçirileceğini belirtir. İki olasılık vardır; Birincisi argümanlar soldan başlanarak sağa doğru fonksiyona geçirilir. İkincisi ise C/C++ dilinde kullanılır ve **argümanlar sağdan başlanarak sola doğru fonksiyona geçirilir**.

Buna örnek olarak aşağıdaki verilen kod örneğindeki durumu inceleyelim;

```
#include <iostream>
void xyzYaz(int x, int y, int z) {
    std::cout << x << " " << y << " "<< z << std::endl;
}
int main() {
    int a = 1;
    xyzYaz(a, ++a, a++);
}
```

Bu kodun çıktısı `1 2 3` olarak beklenir ancak çağrı kuralından ötürü çıktı `3 3 1` olur. **Bunun nedeni C/C++ dilinin çağrı kuralının sağdan sola olmasıdır.** Bunu şöyle açıklayabiliriz;

1. Fonksiyona sağdan ilk argüman olarak `a` değişkeninin değeri 1 geçirilir.
2. Ardından `a++` ifadesi ile `a` değişkeni 2'ye yükseltilir.
3. Sonra `++a` ifadesi ile `a` değişkeni 3'e yükseltilir.
4. Fonksiyona ikinci argüman olarak 3 geçirilir.
5. Fonksiyona son olarak `a` değişkeninin son değeri olan 3 geçirilir.

7.5 Depolama Sınıfları

Depolama sınıfları (storage classes) bir değişkenin ömrünü (lifetime), görünürlüğünü (visibility), bellek konumunu (memory location) ve başlangıç değerini (initial value) belirlemek için kullanılır. Bu özellikler temel olarak bir programın çalışma süresi boyunca belirli bir değişkenin varlığını izlememize yardımcı olan **kapsam** (scope) görünürlüğü ve yaşam süresini içerir.

Yapısal programlama ve fonksiyon kavramının bilgisayarlarda kullanılmasıyla birlikte işlemcilerde de değişiklikler olmuştur. Şekil 1.3'de anlatılan işlemciye yeni kaydediciler de eklenmiştir;

- Veriler ile icra edilen kod farklı bellek bölgesinde bulunur. Bu nedenle verileri işaret eden **veri adresleri kaydedicisi** (data memory register) işlemciye eklenmiştir.
- **Yığın bellek kaydedicisi** (stack register), fonksiyon çağrıları sırasında mevcut işlemci durumu ve yerel değişkenler gibi bazı verileri depolamak için kullanılan belleği gösteren (stack pointer) adresi tutan kaydedici işlemciye eklenmiştir.
- **Bayrak kaydedicisi** (flag register), işlemcinin durumu veya sıfır (zero) bayrağı, taşıma (carry) bayrağı, işaret (sign) bayrağı, taşma (overflow) bayrağı, eşlik (parity) bayrağı, yardımcı taşıma (auxiliary carry) bayrağı ve kesme etkinleştirme (interrupt enable) bayrağı gibi çeşitli işlemlerin sonucunu tutmak için kullanılan özel bir kaydedici işlemciye eklenmiştir.

- ◇ **Genel amaçla kullanılan kaydediciler** (general purpose registers) işlemciye eklenmiştir.

Eklenen bu kaydediciler ile derlenen her bir program, dört bellek bölgesinden (segment) oluşur;

1. **Kod bellek**(code segment) ya da kaynak koda ithafla **metin bellek** (text segment) icra edilecek emirleri içeren bellektir.
2. Programın işleyeceği verileri tutan **veri belleği** (data segment).
3. Yığın bellek (stack segment), fonksiyon çağrıları sırasında kullanılan bellektir. Bu bellek, son giren veri ilk çıkar LIFO (last in first out) mantığıyla çalışır. Çünkü;
   - ◇ **Çağrıdan** önce mevcut işlemci durumu ve yerel değişkenler gibi bazı verileri depolamak ve
   - ◇ Fonksiyondan **dönülünce** işlemciyi ve çağrı ortamını eski hale getirmek için kullanılır.
4. Programın icrası sırasında dinamik olarak eklenip çıkarılan veriler için hurdalık gibi kullanılan **öbek bellek** (heap segment).

Depolama sınıfları, tanımlanan değişkenlerin hangi bellek bölgesinde (segment) yer alacağını belirler. Dört tür depolama sınıfı vardır. Aşağıda başlıklar halinde verilmiştir.

§7.5.1 auto Depolama Sınıfı

Otomatik depolama sınıfı (auto storage class), bir fonksiyon veya blok içinde kimliklendirilmiş tüm yerel değişkenler için varsayılan (default) depolama sınıfıdır.

> C++ dilinde bu depolama sınıfı için değişkenlerde `auto` sıfatı C++11'e kadar kullanılmıştır. C++11 ve sonrasında `auto` anahtar kelimesi ileride anlatılacak **tip çıkarımında** kullanılmaya başlanmıştır. Kısaca `auto` anahtar kelimesi **depolama sınıfı amacıyla kullanılmaz**!

Otomatik değişkenlere; **Yalnızca tanımlı olduğu blok içinden erişilebilir**. Yani değişkenin kapsamı, içinde bulunduğu blok ile sınırlıdır. Bu değişkenler, yığın bellek bölgesinde (stack segment) tutulur.

```
#include <iostream>
int main() {
    int a = 32;
    int b=20;
    /* a ve b değişkenleri bu fonksiyon bloğu ve iç bloklarda geçerlidir. Yani faaliyet alanları
    ↪ (scope) bu fonksiyon bloğu ile sınırlıdır.*/
    if (a==32) {
        int b=10; // Ana fonksiyon bloğunda tanımlı b değişkenine artık ulaşılamaz.
        int c=50; /* c değişkeni if bloğunda ve tanımlanabilecek iç bloklarda geçerlidir.*/
        a=16;// Bu blokta a da geçerlidir.
        std::cout << "a:" << a << " b:" << b << " c:" << c << std::endl;
    }
    /*if bloğu dışında sadece ana fonksiyon bloğunda tanımlı değişkenlere ulaşılabilir. */
    std::cout << "a:" << a << " b:" << b;
}
/*Program Çıktısı:
a:16, b:10 c:50
a:16, b:20
*/
```

§7.5.2 static Depolama Sınıfı

Statik depolama sınıfı (static storage class) yaygın olarak kullanılan statik değişkenleri (static variable) saklamak için kullanılır. Bu değişkenler, `static` sıfatı (static specifier) ile tanımlanır. **Statik** olarak tanımlanan değişken, program başladığında veri bellek bölümünde oluşturulup program bitene kadar aynı bellek bölgesini işgal eden değişkendir;

- ◇ Statik değişkenlere **Yalnızca tanımlı olduğu blok içinden erişilebilir**. Yani değişkenin kapsamı, içinde bulunduğu blok ile sınırlıdır.
- ◇ Statik değişken kodun yazılı olduğu dosyanın başında tanımlı ise, bu değişkene dosya içerisinde her yerden erişilebilir.
- ◇ Statik değişkenler **kapsam (scope) dışına çıktıktan sonra bile değerlerini koruma özelliğine sahiptir**.
- ◇ Statik değişkenlere veri bellek bölgesinde (data segment) yer tahsis edilir. Derleme sırasında bu bellek hep sıfırla doldurulduğundan ilk değerler (initial value) hep sıfırdır.

```
#include <iostream>
void func() {
    for (int sayac = 1; sayac < 10; sayac++) {
```

```
        static int statikOlan = 5; //statik değişken tanımı. İlk değer vermeseydik değeri 0
        ↪ olurdu.
        int statikOlmayan = 10;
        statikOlan++;
        statikOlmayan++;
        std::cout << "sayaç: " << sayac << ", staticOlan: " << statikOlan << std::endl;
        std::cout << "sayaç: " << sayac << ", statikOlmayan: " << statikOlmayan << std::endl;
    }
    //Burada statikOlan değişkene erişilemez.
}
int main() {
    func();
}
/* Program çalıştırıldığında:
Sayaç: 1, staticOlan: 6
sayaç: 1, statikOlmayan: 11
sayaç: 2, staticOlan: 7
sayaç: 2, statikOlmayan: 11
sayaç: 3, staticOlan: 8
sayaç: 3, statikOlmayan: 11
sayaç: 4, staticOlan: 9
sayaç: 4, statikOlmayan: 11
*/
```

§7.5.3 extern Depolama Sınıfı

Harici depolama sınıfı (extern storage class) bize basitçe değişkenin kullanıldığı blokta değil başka bir yerde tanımlandığını söyler;

- Harici tanımlanan değişkenlere değerler tanımlandığı yerde değil farklı bir yerde atanır ve üzerinde değişiklik yapılabilir.
- Bir değişkene `extern` sıfatı (extern specifier) ile tanımlandığında evrensel (global) değişken olur. Böyle tanımlanan değişkenlere programın her yerinden erişilebilir.
- Harici tanımlanan değişkenler de veri bellekte (data segment) yer alır. Derleme sırasında bu bellek hep sıfırla doldurulduğundan ilk değerler hep sıfırdır.

Bir harici değişkene `extern` sıfatı verildiğinde **harici değişken** (extern variable) adını alır ve bir başka dosyada tanımlanmış olduğu kabul edilir. Buna örnek olarak aşağıdaki iki farklı kaynak kod dosyası gereklidir. Birinci kaynak kod dosyası (`xtrn.cpp`) aşağıda verilmiştir.

```
//EXTERN.CPP
int externDegisken; //Diğer dosyalarda kullanılacak değişken tanımlandı.
void funcExtern() //Diğer dosyalarda kullanılacak fonksiyon tanımlandı.
{
    externDegisken++;
}
```

Bu dosyadaki değişken ve fonksiyonları kullanan bir başka kaynak kod `main.cpp` aşağıda verilmiştir;

```
//MAIN.CPP
#include <iostream>
extern void funcExtern(); // harici fonksiyon. Burada bildirim yapıldı. Başka dosyada tanımlandı.
int main() {
    extern int externDegisken; // harici değişken. Burada bildirim yapıldı. Tanımı başka dosyada!
    ↪ Aslında aynı değişkene evrensel olarak erişebiliyoruz!
    funcExtern();
    std::cout << "extern: " << externDegisken << std::endl;
    externDegisken = 2;
    std::cout << "extern: " << externDegisken << std::endl;
}
```

Bu iki dosyayı birlikte derlemek için `g++ xtrn.cpp main.cpp -o main.exe` komutu ile derleyiciye iki farklı dosya parametre olarak verilir.

§7.5.4 register Depolama Sınıfı

Kaydedici depolama sınıfı (register storage class), otomatik değişkenlerle aynı işlevselliğe sahiptir. Tek fark, derleyicinin, eğer boş bir işlemci kaydedicisi mevcutsa değişken olarak o kaydedicinin kullanılmasıdır. Genel amaçlı kaydediciler bu amaçla kullanılır. Bu da bu değişkenlerin, bellekte saklanan değişkenlerden çok daha hızlı olmasını sağlar. Bu değişkenler `register` sıfatı (register specifier) ile tanımlanırlar;

- Eğer boş bir CPU kaydedicisi mevcut değilse, değişken yalnızca bellekte saklanır.
- Sadece bloklar içinde tanımlanan yerel (local) değişkenler `register` ile tanımlanır. Evrensel (global) değişkenler bu anahtar sözcük ile kullanılamaz.

Aşağıda kaydedici bu depolama sınıfına ilişkin örnek verilmiştir. Buradaki sayma ve toplama işlemleri uygun olan işlemci kaydedicilerinde yapılır.

```
#include <iostream>
int main() {
    register int k, sum; //kaydedici değişkeni
    for(k = 1, sum = 0; k < 1000; sum += k, k++);  //döngü bloğu olmayan for talimatı örneği
        std::cout << sum << std::endl;
}
/*Program Çıktısı:
499500
*/
```

C++ standardı gereği, tek bir değişkenle birden fazla depolama sınıfının kullanılmasını yasaktır!
Aynı değişkene hem `static` hem `extern` sıfatı kullanılamaz.

Depolama sınıfları aşağıdaki tabloda özetlenmiştir; Tablodaki **çöp** (garbage) kavramı tahsis edilen bellek içeriği o anda ne ise değişkenin o değeri alması anlamındadır.

Depolama sıfatı	Bellek Bölgesi	Başlangıç Değeri	Değişken Kapsamı (scope)	Değişken Ömrü (lifetime)
`auto`	Yığın bellek (stack segment)	Çöp	Bulunduğu blok	Bulunduğu blok süresince
`static`	Veri bellek (data segment)	Sıfır	Bulunduğu Blok veya Bulunduğu Dosya	Program süresi
`extern`	Veri bellek (data segment)	Sıfır	Evrensel (global)	Program süresi
`register`	İşlemci Kaydedicileri	Çöp	Bulunduğu blok	Bulunduğu blok süresince

Tablo 7.4: Depolama Sınıfları

7.6 Evrensel ve Yerel Değişken

C++ bir değişkene her yerden erişilmek isteniyorsa, bu değişkenlere **evrensel değişken** (global variable) denir. Bu değişkenler herhangi bir kod bloğu içinde tanımlanmazlar. Genel olarak kaynak kodun başında tanımlanırlar. Sonra kodun her yerinde kullanılabilirler. Eğer kodumuz birden çok kaynak kod dosyasından oluşacak ise diğer tüm dosyalarda bu değişken `extern` sıfatı ile tanımlanır.

Blok içinde tanımlanmış tüm değişkenler, o blok içinden erişilebilen **yerel değişkenlerdir** (local variable). Yerel değişkenler otomatik depolama sınıfında (auto storage class) yer alır ve hepsi yığın belleği (stack segment) işgal eder.

```
1  #include <iostream>
2  int evrenselDegisken=10; /* evrenselDeğişken bu noktadan sonra her yerde kullanılabilir. */
3  void fonksiyon();
4  int main() {
5      int yerelDegisken=20; /* Bu yerel değişken sadece main bloğu içinde kullanılabilir.*/
6      std::cout << "evrensel:" << evrenselDegisken << ", yerel: "
7      << yerelDegisken << std::endl;
8      evrenselDegisken++; //evrensel değişken değiştiriliyor.
9      yerelDegisken--; //yerel değişken değiştiriliyor.
10     std::cout << "evrensel:" << evrenselDegisken << ", yerel:"
11     << yerelDegisken << std::endl;
```

```
12      fonksiyon();
13  }
14  void fonksiyon() {
15      int fonksiyonYerelDegiskeni=30;
16      evrenselDegisken++;
17      std::cout << "evrensel:"<< evrenselDegisken << ", fonsiyonYerel:"
18      << fonksiyonYerelDegiskeni << std::endl;
19      if (fonksiyonYerelDegiskeni==30) {
20          int evrenselDegisken=100; //Artık evrensel değişkene bu blokta ulaşılamaz.
21          std::cout << "evrensel:"<< evrenselDegisken << ", fonsiyonYerel:"
22          << fonksiyonYerelDegiskeni << std::endl;
23      }
24  }
25  /* Program Çıktısı:
26  evrensel:10, yerel:20
27  evrensel:11, yerel:19
28  evrensel:12, fonsiyonYerel:30
29  evrensel:100, fonsiyonYerel:30
30  */
```

Burada fonksiyon içinde `if` bloğunda 18. satırda yer alan değişken her ne kadar `evrenselDegişken` olarak kimliklendirilse de yerel değişkendir ve sadece bu `if` bloğu içinde geçerlidir. Bu yerel değişken 2. satırdaki değişkeni gölgelemiştir. Yani tanımlamada **mantıksal hata** (logical error) yapılmıştır.

7.7 Değerle Çağırma

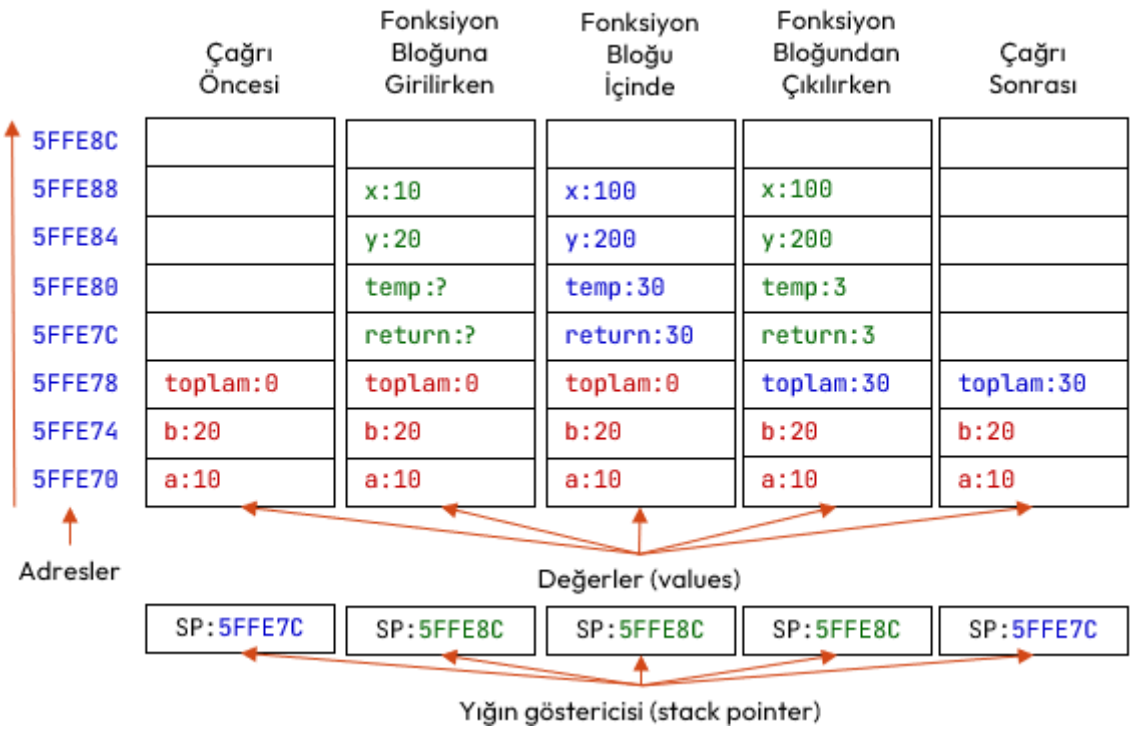
Yerel değişkenlerin yığın bellekte tutulduğu bir üst başlıkta anlatılmıştı. Fonksiyon blokları içindeki değişkenler de yerel değişkenlerdir. Bir fonksiyon çağrıldığında değişkenlerin bellekteki durumu ve değişimi aşağıdaki programdan anlaşılabilir;

```
#include <iostream>
int topla(int, int); /* İki tam sayıyı toplayan fonksiyon bildirimi/prototipi */
int main () {
    /* main() fonksiyonu içinde geçerli yerel (local) değişkenler*/
    int a = 10;
    int b = 20;
    int toplam = 0;
    std::cout << "çağrı öncesi a:" << a << ", b:" << b << ", toplam:" << toplam << std::endl;
    toplam = topla(a, b);
    std::cout << "çağrı sonrası a:"<< a << ", b:" <<b << ", toplam:" << toplam << std::endl;
}
int topla(int x, int y) { /* iki tam sayıyı toplayan fonksiyon tanımı*/
    /* Topla fonksiyonu yerel değişkenler */
    int temp= x+y;
    x=100; y=200;
    /*
    Burada değiştirilenler parametre değişkenleridir.
    main() içindeki yerel değişkenler değildir.
    */
    std::cout << "çağrı içinde x:" << x << ", y:" << y  << std::endl;
    return temp;
}
/* Program çalıştırıldığında:
çağrı öncesi a:10, b:20 toplam:0
çağrı içinde x:100, y:200
çağrı sonrası a:10, b:20 toplam:30
*/
```

Yukarıda örneği verilen programda `topla` fonksiyonunun çağırma sürecindeki yığın bellek bölgesindeki (stack segment) değişimi Şekil 7.3'de gösterilmektedir.

Yukarıdaki kod şu şekilde açıklanabilir;

- İcra sırası `topla(a,b)` fonksiyonuna geldiğinde fonksiyon çağrılmadan önce `main()` fonksiyonu yerel değişkenleri olan `a`, `b` ve `toplam` değişkenleri yığın belleğe (stack segment) itilmiştir (push).



Şekil 7.3: Fonksiyon Çağırma Sürecinde Yığın Bellek

- `topla(a,b)` fonksiyonu çağrıldığı anda `a` ve `b` **değişkenlerinin değerleri**, `x` ve `y` **parametrelerine kopyalanarak fonksiyona geçirilir**. Değerler parametrelere kopyalandığından buna **değerle çağırma** (call by value) adı verilir.
- `topla(a,b)` fonksiyonu icra edilirken fonksiyon yerelindeki `temp` ve parametre değişkenleri `x`, ve `y` istenildiği gibi değiştirilebilir. Geri dönüş değeri de belirlenir.
- `topla(a,b)` fonksiyonu içine bir başka fonksiyon da çağrılabilirdi. Böyle bir durumda `x`, `y` ve `temp` değerleri yığın belleğe itilir ve çağrılan fonksiyon icra edilirdi. Böylece iç içe çağrılan her fonksiyonda yığın bellek büyüyeceği görülmektedir. Program derlendiğinde yığın bellek (stack segment) için sınırlı bir bellek bölgesi ayrılır. İç içe çok fonksiyon çağrıldığında bu bellek sınırı aşılabilir. Bu durumda program **yığın taşması** (stack overflow) hatası verecektir.
- `toplam=topla(a,b)` fonksiyon bloğundan çıkılırken geri dönüş değeri çağrı ortamındaki `toplam` değişkenine kopyalanır ve fonksiyon için yığın belleğe itilen değerler geri çekilir (pop).

7.8 Özyinelemeli Fonksiyonlar

Özyineleme (recursion), bir fonksiyonun kendi çağrısını yapma tekniğidir. Bu teknik, karmaşık sorunları çözülmesi daha kolay basit sorunlara ayırmanın bir yolunu sağlar. Özyineleme, yineleme (iteration), döngüler (loop) veya tekrar yerine geçebilecek çok güçlü bir tekniktir. Özyinelemeli algoritmalarda, tekrarlar fonksiyonun kendi kendisini kopyalayarak çağırması ile elde edilir. Bu kopyalar işlerini bitirdikçe kaybolur.

Bir problemi özyineleme ile çözmek için problem iki ana parçaya ayrılır;

1. Cevabı kesin olarak bildiğimiz temel durum (base case)
2. Cevabı bilinmeyen ancak cevabı yine problemin kendisi kullanılarak bulunabilecek durum.

Faktöriyel örneğini inceleyelim; Matematikte, negatif olmayan bir tam sayı olan n sayısının faktöriyeli, $n!$ ile gösterilir ve n sayısına eşit ve küçük olan tüm pozitif tam sayıların çarpımıdır.

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 3 \times 2 \times 1$$

Sıfır sayısı da pozitif bir sayıdır ama sonucun sıfır çıkmaması için **sıfır sayısının faktöriyeli 1 kabul edilir**. Bu aslında faktöriyel problemi için **temel durumdur** (base case) ve **boş ürün** (emty product) kuralı olarak bilinir. Aşağıda faktöriyel için girilen sayıdan sonra tüm sayıların çarpımıyla hesaplayan bir fonksiyon yazılmıştır.

```
#include <iostream>
unsigned int faktoriyel(unsigned int n) { //Fonksiyon tanımı yapıldı. Bildirim yapılmadı.
    int sonuc=1,indis;
```

```
    for (indis=n;indis>0;indis--)
        sonuc=sonuc*indis;
    return sonuc;
}
int main() {
    unsigned int sayi,sonuc; //Pozitif tam sayı tanımı yapılmış değişkenler.
    std::cout << "Pozitif Bir Sayi Giriniz:";
    std::cin >> sayi;
    sonuc=faktoriyel(sayi);
    std::cout << sayi << "!=" << sonuc;
}
```

Aslında negatif olmayan bir tam sayı olan sayının faktöriyeli, kendisi ile kendisinden bir küçük olan sayının faktörüyeli ile çarpımı şeklinde yazılabilir. Yani problemin tanımı yine kendisini içerir:

$$n! = n \times (n-1)!$$

Bu durumda faktöriyel fonksiyonu aşağıdaki şekilde **özyinelemeli** (recursive) tanımlanabilir;

$$n! = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot (n-1)! & \text{eğer } n > 0 \end{cases}$$

Aynı faktöriyel fonksiyonu **parametreli olarak** aşağıdaki şekilde yine özyinelemeli olarak tanımlanabilir;

$$f(n) = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot f(n-1) & \text{eğer } n > 0 \end{cases}$$

Buna örnek olarak aşağıdaki hesabı verebiliriz;

$$5! = 5 \times 4! = 5 \times 4 \times 3! = 5 \times 4 \times 3 \times 2! = 5 \times 4 \times 3 \times 2 \times 1! == 5 \times 4 \times 3 \times 2 \times 1 \times 0! = 120$$

Özyinelemeli (recursive) olarak ise faktöriyel fonksiyonu aşağıdaki şekilde kodlanabilir;

```
#include <iostream>
unsigned int faktoriyel(unsigned int n)
{
    if (n==0)
        return 1;
    else
        return n*faktoriyel(n-1);
}
int main() {
    unsigned int sayi,sonuc; //Pozitif tam sayı tanımı yapılmış değişkenler.
    std::cout << "Pozitif Bir Sayi Giriniz:";
    std::cin >> sayi;
    sonuc=faktoriyel(sayi);
    std::cout << sayi << "!=" << sonuc;
}
```

Bir başka örnek olarak, girilen taban ve üs ile kuvvet hesaplama fonksiyonu verilebilir. Bu fonksiyonun tanımı aşağıdaki şekilde yazılabilir;

$$f(taban, us) = \begin{cases} 0 & \text{eğer } taban = 0 \\ 1 & \text{eğer } us = 0 \\ taban \cdot f(taban, us-1) & \text{eğer } taban > 0 \text{ ve } us > 0 \end{cases}$$

Buna ilişkin kod aşağıda verilmiştir;

```
#include <iostream>
int Kuvvet(int taban, int us) {
    if (taban == 0)
        return 0; //Taban 0 ise 0 dön
    if (us == 0)
        return 1;  // Üs 0 ise 1 dön
    return taban* Kuvvet(taban, us - 1);
}
int main(){
    int taban,us;
    std::cout << "Tabanı Giriniz:";
```

```
    std::cin >> taban;
    std::cout << "Üssü Giriniz:";
    std::cin >> us;
    std::cout << taban <<" uzeri "<< us <<" = " << Kuvvet(taban,us);
}
/*Program Çıktısı:
Tabanı Giriniz:3
Üssü Giriniz:7
3 uzeri 7 = 2187
*/
```

7.9 Sabit İfadelerin Fonksiyonlarda Kullanımı

Sabit ifadeler (const expression), 4.6 başlığında işlenmişti. C++11 ile gelmiş ve modern C++'ın en güçlü özelliklerinden biri olan bu ifadeler `constexpr` anahtar kelimesini kullanır. Temel mantığı şudur: "Eğer bir hesaplamayı program çalışırken değil de derlenirken yapabiliyorsak, yapalım; böylece program daha hızlı çalışır. Aşağıda fonksiyonlardaki kullanımına örnek verilmiştir.

```
constexpr int kareAl(int n) {
    return n * n;
}
int main() {
    // Derleyici buraya doğrudan 25 yazar, fonksiyonu çağırmakla vakit kaybetmez.
    int dizi[kareAl(5)];
}
```

7.10 Satır İçi Fonksiyonlar

Fonksiyon Çağırma Süreci başlığında anlatıldığı üzere fonksiyonlar çağrılırken sürekli yığın belleğe (stack segment) it-çek (push-pop) yapılır. Bazı durumlarda bu işlem performans gereği istenmez. Bu durumda fonksiyonlar, **satır içi fonksiyon** (inline function) olarak tanımlanır. 2017 C sürümü ile gelen bu özellik, fonksiyona `inline` sıfatı (inline specifier) eklenerek kullanılır.

```
#include <iostream>
inline int topla (int op1, int op2) {
    int toplam= op1+op2;
    return toplam;
}
int main() {
    int x=10, y=15, sonuc;
    sonuc = topla(x,y);
    std::cout << x << "+" << y << "=" << sonuc;
}
```

Bu kod aşağıdaki şekle çevrilmiş gibi derlenir;

```
#include <iostream>
int main() {
    int x=10, y=15, sonuc;
    {
        int toplam= op1+op2;
        sonuc = toplam;
    }
    std::cout << x << "+" << y << "=" << sonuc;
}
```

Bir fonksiyon **aşağıda belirtilen durumlarda satır içi fonksiyon olarak derlenmez;**

1. Bir döngü içeriyorsa!
2. Statik değişkenler içeriyorsa!
3. Özyinelemeli ise!
4. Dönüş türü `void` haricinde olup `return` ifadesi işlev gövdesinde mevcut değilse!
5. `switch` veya `goto` talimatı içeriyorsa!

Bu tip fonksiyon tanımlamadaki amacımız, **fonksiyon çağırma yükünün azaltılması için işlemci kaydedi-**

cilerinin daha çok kullanılmasıdır. Bu da icra hızını yükseltir.

7.11 Varsayılan Argümanlar

Varsayılan argüman (default argument), bir fonksiyon bildiriminde bir parametre için sağlanan ve çağıran fonksiyon bu parametreler için bir değer sağlamazsa derleyici tarafından otomatik olarak atanan bir değerdir. Bu parametreye çağrı sırasında bir değer geçirilirse, varsayılan değer dikkate alınmaz.

```
#include <iostream>
void yaz(int a = 10) { //a parametresi için varsayılan argüman 10 dur.
    std::cout << a << std::endl;
}
int main() {
    yaz(); // 10
    yaz(200); //200
}
```

Varsayılan argüman fonksiyon bildiriminde (function declaration) farklı, fonksiyon tanımında (function definition) farklı verilemez!

```
#include <iostream>
void yaz(int a = 10); //a parametresi için varsayılan argüman 10 dur.
int main() {
    yaz(); // 10
    yaz(200); //200
}
void yaz(int a = 20); // Derleme hatası olur. 10 olarak verilmelidir.
{
    std::cout << a << std::endl;
}
```

Birden fazla parametreye varsayılan argüman atanacak ise sağdan sola doğru hepsine atanmalıdır;

```
void yaz(int x, int y = 20); // Geçerli Bildirim.
void yaz(int x = 10, int y); /* Derleme hatası olur. GEÇERSİZ Bildirim: sağdaki y parametresine
↪ varsayılan argüman atanmadı */
```

7.12 Aşırı Yükleme

C++ dilinde **aşırı yükleme** (overloading); fonksiyonların farklı parametrelerle yeniden tanımlanmasıdır. **Aşırı yüklemede fonksiyonun kimliği ve geri dönüş tipi değişmez!** Farklı argümanlar ile fonksiyon gövdesi yeniden tanımlanır. Aşağıda bir fonksiyonun aşırı yüklemesine bir örnek verilmiştir.

```
#include <iostream>
void yaz(int a, int b) {  //İlk tanımlanan yaz fonksiyonu
    std::cout << "toplam = " << (a + b);
}
void yaz(double a, double b) {  // Aşırı yüklenmiş yaz fonksiyonu
    std::cout << std::endl << "toplam = " << (a + b);
}
int main() {
    yaz(1, 2);
    yaz(2.5, 5.5);
}
```

Aşırı yüklemede argümanların farklı tanımlandığının derleyiciye bildirilmesi gerekir. Aksi durumda hata alınır;

```
int topla(int);
int topla(int x);  //Hata!
int topla(int y);  //Hata!
int main(){
    topla(1);
}
int topla(int i) {
    return i+10;
}
int topla(int x) { //Hata: redefinition of 'int topla(int)'
```

```
    return x+20;
}
int topla(int y) { //Hata redefinition of ‘int topla(int)’
    return y+20;
}
```

Fonksiyonlarda ön tanımlı argümanlar kullanıldığında aşırı yükleme tanımlanmasına çok dikkat edilmelidir;

```
int topla(int a = 10, int b = 20) {
    return a+b;
}
int topla(int a = 10, int b = 0) { // HATA: fonksiyon imzası (signature) aynı!
    // redefinition of ‘int topla(int, int)’
    return a+b;
}
int topla(int a = 10) { // HATA: üstteki fonksiyon da çağrıyı karşılar
    // call of overloaded ‘topla(int)’ is ambiguous
    return a+1;
}
int main()
{
    topla(1,2)
    topla(1);
}
```

7.13 Düşün ve Çöz

- **Düşün**: `static` bir yerel değişken ile `auto` bir yerel değişkenin bellek ömrü ve kapsamı arasındaki farkları karşılaştırınız.
- **Düşün**: Fonksiyonlara parametre aktarırken değerle çağırma (call by value) yönteminin yığın bellek kullanımına etkisini açıklayınız.
- **Düşün**: Özyinelemeli fonksiyonlarda temel durum (base case) unutulursa bellek seviyesinde ne gerçekleşir? Yığın taşması (stack overflow) nedir?
- **Çöz**: Bir tam sayının basamak değerleri toplamını özyinelemeli olarak hesaplayan fonksiyonu yazınız.
- **Çöz**: `extern` anahtar kelimesini kullanarak, iki farklı kaynak dosyasında (`.cpp`) ortak kullanılan bir evrensel değişken örneği kurgulayınız.
- **Düşün**: Değerle geçirme (pass by value) ile referansla geçirme (pass by reference) arasındaki farkı bellek diyagramı çizerek açıklayın. Büyük bir nesneyi fonksiyona geçirirken hangisi tercih edilmeli ve neden?
- **Düşün**: Özyinelemeli (recursive) bir fonksiyon ile aynı işi yapan döngüsel (iterative) çözümü karşılaştırın. Fibonacci sayı dizisini her iki yöntemle hesaplayın ve yığın (stack) kullanımı açısından değerlendirin.
- **Düşün**: Satır içi fonksiyon (inline function) kullanmanın avantajları nelerdir? Derleyici her zaman `inline` isteğini yerine getirir mi? Hangi durumlarda `inline` kullanımı anlamsız olur?
- **Düşün**: Depolama sınıfları (`auto`, `static`, `extern`, `register`) arasındaki farkları bir tablo halinde özetleyin. `static` bir yerel değişken ile global değişken arasındaki fark nedir?
- **Düşün**: Kaynak dosyada aynı fonksiyon adını birden fazla tanımlamak ne tür bir hataya yol açar? Bu hatayı `extern` ve başlık dosyaları kullanarak nasıl çözersiniz?